SOURCE CODE & REPOSITORY MANAGEMENT

---

1. Overview

The E-Commerce application uses GitHub as the centralized version control platform for managing source code, team collaboration, pull requests, code reviews, and release history.

All microservices source code, Kubernetes manifests, and deployment configurations are maintained in Git repositories to ensure:

- Proper version tracking
- Controlled change management
- Rollback capability
- Secure collaboration
- Streamlined development workflows

---

2. GitHub Repository Details

2.1 Main Application Repository

Repository URL:

https://github.com/e-commerce-application-microservice/e-commerce-app.git

Repository Contains:

- Frontend source code
- Backend microservices
- API Gateway
- Dockerfiles
- CI pipeline files
- Application configuration files

2.2 GitOps / Deployment Repository

Repository URL:

https://github.com/e-commerce-application-microservice/ecommerce-argo-cd

Repository Contains:

- Helm charts
- Kubernetes manifests
- Argo CD application definitions
- Environment values files
- Rollout strategies
- Deployment automation files

---

3. Branch Strategy

A structured branching model is implemented to support parallel development, infrastructure changes, security enhancements, and stable production releases.

3.1 main

Primary stable branch used for production-ready code.

Used For:

- Final tested releases
- Approved code changes only
- Production deployments
- Stable application versions

3.2 RBAC

Dedicated branch for Kubernetes security and access control configurations.

Used For:

- role.yaml
- rolebinding.yaml
- service accounts
- namespace permissions

Purpose:

Implements Role-Based Access Control (RBAC) for secure Kubernetes operations.

3.3 feature/argo-blue-green

Feature branch used for progressive deployment strategy implementation.

Used For:

- Argo Rollouts configuration
- Blue-Green deployment manifests
- Traffic switching setup
- Rollback strategy

Purpose:

Enables zero-downtime deployments with controlled traffic switching.

3.4 feature/ci-pipeline

Feature branch used for Continuous Integration automation.

Used For:

- GitHub Actions workflows
- Docker image build pipeline
- Security scans
- GHCR image publishing

Purpose:

Automates build, test, and image publishing workflows.

---

4. Branch Workflow

Feature Branches
↓
Pull Request
↓
Code Review

↓

main Branch

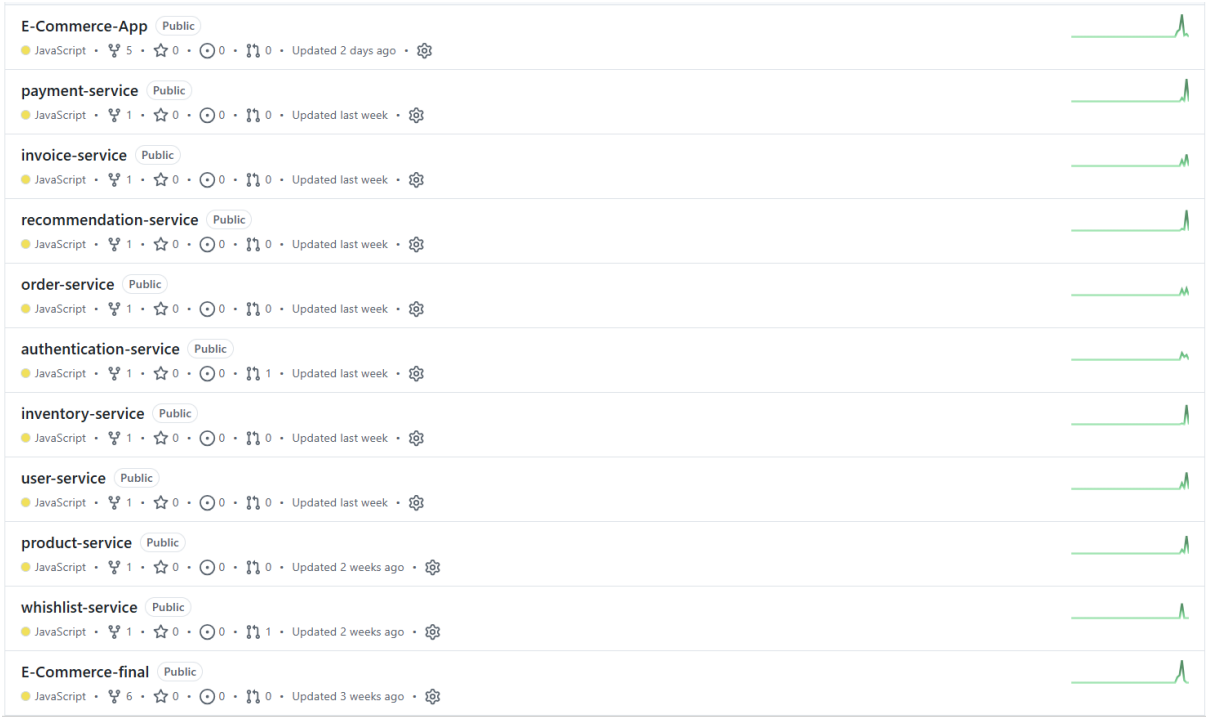
↓

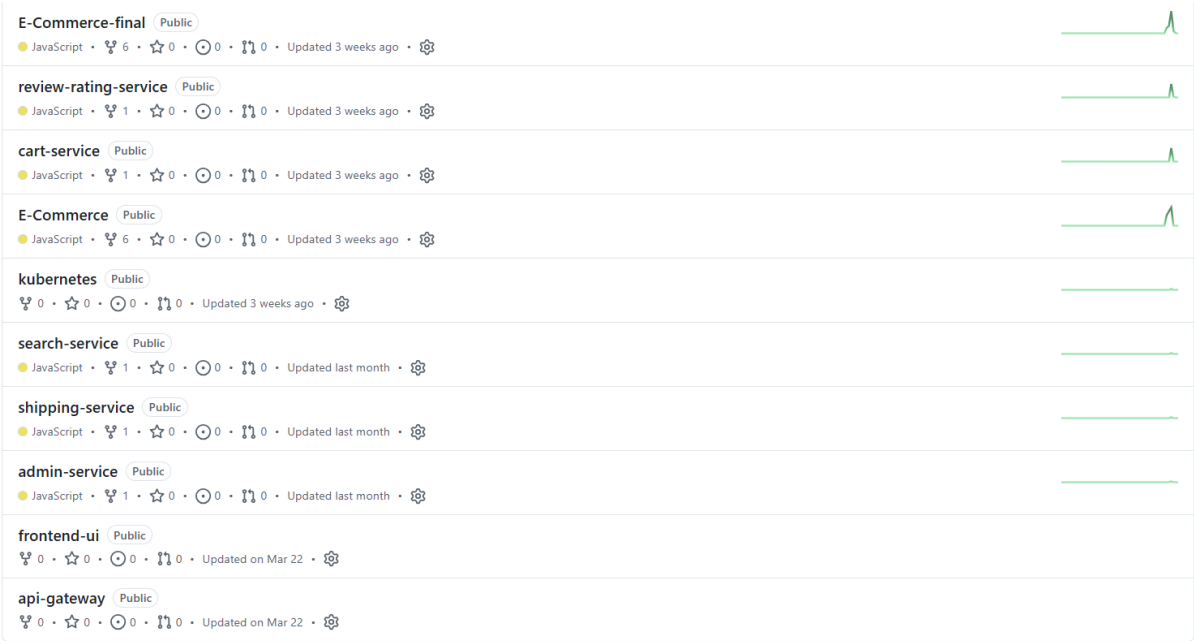
Production Deployment

---

5. Microservices Overview

The application follows a microservices architecture where each service is independently developed and deployed to handle a specific business capability.





---

6. Microservices and Port Configuration

S.No	Service Name	Type	Cluster IP	External IP	Port(s)
1	admin-service	ClusterIP	None		3014/TCP
2	api-gateway	ClusterIP	10.102.149.140		3000/TCP
3	auth-service	ClusterIP	None		3002/TCP
4	cart-service	ClusterIP	None		3006/TCP
5	ecommerce-gateway	LoadBalancer	10.104.235.220		80:32367/TCP
6	inventory-service	ClusterIP	None		3005/TCP

7	invoice-service	ClusterIP	None		3010/TCP
8	mongodb	ClusterIP	None		27017/TCP
9	order-service	ClusterIP	None		3008/TCP
10	payment-service	ClusterIP	None		3009/TCP
11	product-service	ClusterIP	None		3003/TCP
12	recommendation -service	ClusterIP	None		3013/TCP
13	review-rating-service	ClusterIP	None		3012/TCP
14	search-service	ClusterIP	None		3004/TCP
15	shipping-service	ClusterIP	None		3011/TCP
16	user-service	ClusterIP	None		3001/TCP
17	wishlist-service	ClusterIP	None		3007/TCP

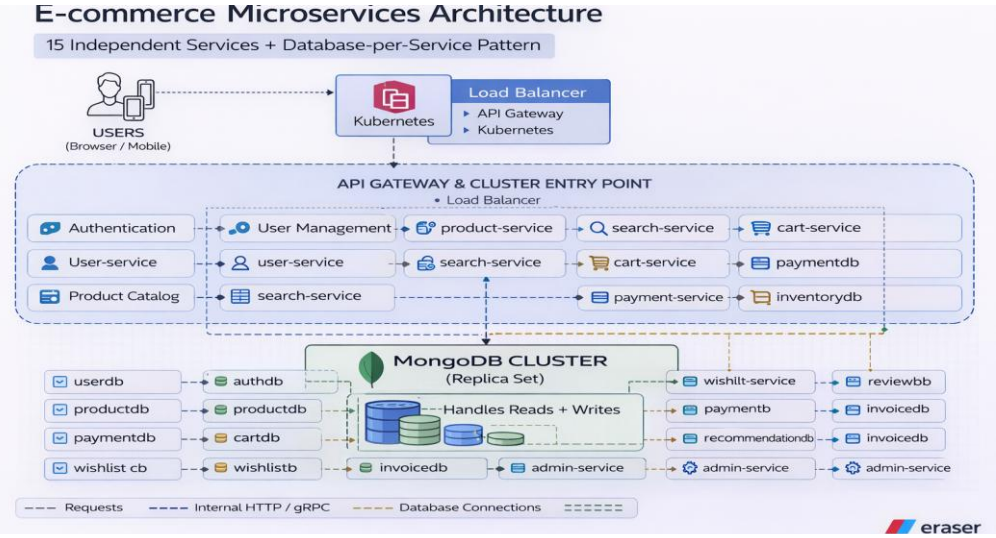
---

7. External Access Configuration

Service Name	NodePort
frontend-ui	30080

---

8. Application Architecture



---

9. Infrastructure Architecture

Developers
↓
GitHub Repositories
↓
CI Pipeline (GitHub Actions)
↓
Container Registry (GHCR)
↓
Argo CD
↓
Kubernetes Cluster
↓
Microservices Pods
↓
Monitoring / Logging / Security Tools

---

10. Core Infrastructure Components

Layer	Tool / Platform	Purpose
Source Control	GitHub	Code management
CI Pipeline	GitHub Actions	Build & test automation
Container Registry	GitHub Container Registry	Store Docker images
CD Tool	Argo CD	Kubernetes deployment
Containerization	Docker	Package applications
Orchestration	Kubernetes	Run containers
Monitoring	Prometheus	Metrics collection
Dashboard	Grafana	Visualization
Logging	Grafana Loki	Centralized logs
Security	Keycloak	Authentication
Ingress / LB	HAProxy	Traffic routing

---

11. Kubernetes Cluster Infrastructure

The application is deployed in a Kubernetes cluster consisting of Control Plane and 2 Worker Nodes.

Control Plane Node Responsibilities

- API Server
- Scheduler
- Controller Manager
- etcd

Worker Node Responsibilities

- Running application pods
- Managing services
- Rollouts
- Scaling workloads

---

12. Application Infrastructure

Frontend UI
↓
API Gateway
↓
15+ Backend Microservices
↓
MongoDB Database

---

13. Deployment Infrastructure

Continuous Integration (CI) Flow

Code Push
→ GitHub Actions
→ Build Docker Image
→ Security Scan
→ Push Image to GHCR

Continuous Delivery (CD) Flow

Image Updated
→ Git Repository Manifest Update
→ Argo CD Detects Change
→ Sync to Kubernetes
→ Deployment Completed